

嵌入式系统设计与开发个人总结报告

2026 年 7 月

目录

1	项目简介	3
2	设计目标与预期成果	3
3	项目设计与实现	3
3.1	硬件设计	3
3.1.1	系统整体架构	3
3.1.2	核心模块选型	4
3.2	软件设计	4
3.2.1	软件架构	4
3.2.2	程序流程图	5
3.2.3	关键代码片段	5
3.2.4	技术实现细节	11
4	物料清单与项目成本	12
4.1	物料清单	12
4.2	项目总成本	12
4.3	损耗与损坏记录	12
5	个人贡献	13
6	项目成果与反思	14
6.1	成果展示	14
6.2	项目评估	14
6.3	个人反思与改进方向	15

7 附录	15
7.1 完整代码清单	15
7.1.1 应用层代码（项目核心源文件）	15
7.1.2 启动文件	16
7.1.3 HAL 驱动层（STM32F1xx HAL 库）	17
7.1.4 FreeRTOS 实时操作系统	17
7.1.5 CMSIS 标准库	18
7.1.6 项目配置文件	18

1 项目简介

项目背景：本项目基于 STM32F103C8T6 平台，开发了一款低功耗运动手表原型。系统集成 MPU6050 六轴惯性传感器、1.3 英寸 OLED 显示屏和 BLE 蓝牙透传模块，并搭载 FreeRTOS 实时操作系统，实现了运动数据的本地采集、实时展示与无线同步上传。

主要功能：设备支持时间显示与运动数据双页面切换，通过带按键的 EC11 旋转编码器完成交互操作；内置基于加速度矢量角度的双沿滞回状态机计步算法，能有效抑制抖动引起的误判；运动数据每 2 秒通过蓝牙上传至 PC 端上位机，上位机可实时显示步数、距离、卡路里和温度，并具备历史记录保存与一键清零功能。

2 设计目标与预期成果

设计目标：本项目旨在设计一款基于 STM32F103C8T6 和 FreeRTOS 的智能手表系统，重点实现三个层面的目标——

- 1. **硬件层面：**完成 MCU 最小系统电路、I2C 总线连接的 OLED 显示屏与 MPU6050 传感器、LDO 电源管理以及 Type-C 接口的锂电池充电电路设计；
- 2. **软件层面：**基于 FreeRTOS 合理划分 UI 显示、时间管理、传感器数据采集等任务，借助信号量与消息队列实现任务间安全通信，确保系统实时性与稳定性；
- 3. **功能扩展层面：**利用旋转编码器实现多级菜单交互，增加蓝牙模块与 PC/手机上位机进行数据同步，并实现运动计步算法与数据记录功能。

预期成果：最终交付硬件样机、完整的嵌入式固件源码（含 FreeRTOS 任务架构）、蓝牙上位机程序及设计文档。系统将实现流畅的 OLED 界面切换、准确的传感器数据读取与计步、稳定的蓝牙无线同步，形成一个功能完整、可扩展的智能手表原型平台。

3 项目设计与实现

3.1 硬件设计

3.1.1 系统整体架构

本项目智能手表采用“核心主控 + 外围模块”的分层架构设计。核心为 STM32F103C8T6 微控制器（ARM Cortex-M3 内核，72MHz 主频，64KB Flash，20KB RAM），通过 I2C、USART、GPIO 等接口连接 OLED 显示屏、MPU6050 六轴传感器、BLE 蓝牙模块、EC11 旋转编码器及电源管理电路。

整体架构划分为四个层次：

- **电源层**：3.7V 锂电池经 LDO 稳压至 3.3V，为所有模块供电；
- **控制层**：STM32F103C8T6 作为核心处理单元，运行 FreeRTOS 实时操作系统；
- **感知与交互层**：MPU6050 采集运动数据，EC11 编码器提供用户输入；
- **通信与显示层**：OLED 显示界面信息，蓝牙模块实现与上位机的无线数据传输。

3.1.2 核心模块选型

表 1: 核心模块选型

模块	型号	关键参数	选型理由
主控	STM32F103C8T6	72MHz, 64KB Flash, 20KB RAM	性能充裕、生态
显示	1.3 寸 OLED (SH1106)	128×64, I2C 接口	高对比度、自发
运动传感器	MPU6050	6 轴 (3 轴陀螺仪 +3 轴加速度计), I2C	集成度高、内置
蓝牙	BLE 串口透传模块	UART 接口, 9600bps	低功耗透传, 与
交互	EC11 旋转编码器	带按键	单键 + 旋转实现
电源管理	LDO + TP4056	3.3V 稳压输出	TP4056 支持锂电
供电	3.7V 锂电池	典型 600mAh	轻薄、容量适中
充电接口	Type-C	支持 5V 输入	通用性强, 便于

3.2 软件设计

3.2.1 软件架构

项目固件采用分层模块化设计，顶层为应用层（主任务、UI、计步、蓝牙等），中间层为 FreeRTOS 内核及 CMSIS-RTOS V2 封装，底层为 STM32 HAL 库及外设驱动。代码目录结构如下（仅列出核心文件）：

```
1 Project/  
2   Core/  
3     Inc/  
4       main.h  
5       oled.h  
6       mpu6050.h  
7       pedometer.h  
8       ui.h  
9       encoder.h  
10      bt.h  
11      cn_font.h
```

```
12         rtc.h
13         i2c.h
14         usart.h
15     Src/
16         main.c
17         freertos.c
18         oled.c
19         mpu6050.c
20         pedometer.c
21         ui.c
22         encoder.c
23         bt.c
24         i2c.c
25         usart.c
26         rtc.c
27         stm32f1xx_it.c
28 Drivers/
29 Middlewares/
30 MDK-ARM/
```

Listing 1: 项目代码目录结构

3.2.2 程序流程图

系统上电后，首先完成时钟、GPIO、I2C、USART、RTC 等底层初始化，随后启动 FreeRTOS 内核，创建初始化任务 (StartDefaultTask) 和主循环任务 (Task_Main)。初始化任务完成编码器、UI 状态机、计步器等模块的初始化后自行删除；主任务以 10ms 周期循环执行输入检测、事件处理、传感器读取、计步运算、界面渲染和蓝牙上传等操作。

3.2.3 关键代码片段

main.c — 系统初始化入口

```
1 main()
2     SystemClock_Config()
3     MX_GPIO_Init()
4     MX_I2C1_Init()
5     MX_USART1_UART_Init()
6     MX_RTC_Init()
7     osKernelInitialize()
8     MX_FREERTOS_Init()
```

```

9         StartDefaultTask
10        Task_Main
11    osKernelStart()

```

Listing 2: main.c 关键函数调用链

freertos.c —FreeRTOS 任务管理

```

1 void MX_FREERTOS_Init(void)
2 {
3     osThreadNew(StartDefaultTask, NULL, &defaultTask_attributes);
4 }
5
6 void StartDefaultTask(void *argument)
7 {
8     Encoder_Init();
9     UI_Init();
10    Pedometer_Init();
11    osThreadNew(Task_Main, NULL, &mainTask_attributes);
12    osThreadTerminate(osThreadGetId());
13 }
14
15 void Task_Main(void *argument)
16 {
17     OLED_Init();
18     OLED_Clear();
19     OLED_ShowString(0, 0, "SportsWatch");
20     MPU6050_Init();
21     BT_Init();
22     while(1)
23     {
24         Encoder_Poll();
25         UI_HandleEvent();
26         MPU6050_Read();
27         Pedometer_Process();
28         Pedometer_GetActivity();
29         HAL_RTC_GetTime(&hrtc, &sTime, RTC_FORMAT_BIN);
30         HAL_RTC_GetDate(&hrtc, &sDate, RTC_FORMAT_BIN);
31         UI_Render();
32         BT_Upload();
33         BT_Process();
34         osDelay(10);

```

```
35     }  
36 }
```

Listing 3: freertos.c 任务创建与主循环

ui.c —UI 状态机架构

```
1  typedef enum {  
2      PAGE_TIME,  
3      PAGE_ACTIVITY,  
4      MENU_TIME,  
5      MENU_ACT,  
6      EDIT_TIME,  
7      EDIT_NUM  
8  } UI_Page;  
9  
10 typedef struct {  
11     UI_Page currentPage;  
12     uint8_t selectedIndex;  
13     uint8_t editValue;  
14 } UI_State_t;  
15  
16 void UI_Init(void)  
17 {  
18     memset(&uiState, 0, sizeof(uiState));  
19     uiState.currentPage = PAGE_TIME;  
20 }  
21  
22 void UI_HandleEvent(void)  
23 {  
24     Event_t event = GetEncoderEvent();  
25     switch (uiState.currentPage) {  
26         case PAGE_TIME:    break;  
27         case PAGE_ACTIVITY: break;  
28         case MENU_TIME:    break;  
29         case MENU_ACT:     break;  
30         case EDIT_TIME:    break;  
31         case EDIT_NUM:     break;  
32     }  
33 }  
34  
35 void UI_Render(void)
```

```

36 {
37     OLED_Clear();
38     switch (uiState.currentPage) {
39         case PAGE_TIME:      PageTime();      break;
40         case PAGE_ACTIVITY: PageActivity(); break;
41         case MENU_TIME:      MenuTime();      break;
42         case MENU_ACT:      MenuAct();      break;
43         case EDIT_TIME:      EditTime();      break;
44         case EDIT_NUM:      EditNum();      break;
45     }
46     OLED_Refresh();
47 }

```

Listing 4: UI 状态机核心代码

pedometer.c — 计步算法核心

```

1 typedef struct {
2     float accelX, accelY, accelZ;
3     float angleX, angleY;
4     uint32_t stepCount;
5     float distance;
6     float calorie;
7     Activity_t activity;
8 } Pedometer_t;
9
10 void Pedometer_Process(void)
11 {
12     // 获取加速度, 计算合加速度, 角度, 峰值检测, 更新步数
13 }
14
15 Activity_t Pedometer_GetActivity(void)
16 {
17     // 根据步频和加速度特征判断运动状态
18 }

```

Listing 5: 计步算法数据结构与处理

mpu6050.c — 传感器驱动

```

1 #define MPU6050_ADDR 0x68
2 #define WHO_AM_I_REG 0x75
3

```

```
4 void MPU6050_Init(void)
5 {
6     // 探测设备、复位、配置电源、量程、滤波器
7 }
8
9 void MPU6050_Read(void)
10 {
11     // 读取寄存器，转换物理值，校准
12 }
```

Listing 6: MPU6050 初始化与读取

oled.c — OLED 显示驱动

```
1 #define OLED_ADDR    0x3C
2 #define OLED_WIDTH   128
3 #define OLED_HEIGHT  64
4 static uint8_t OLED_Buffer[128 * 8];
5
6 void OLED_Init(void)
7 {
8     HAL_Delay(100);
9     OLED_Cmd(0xAE);
10    // 其他初始化命令
11    OLED_Cmd(0xAF);
12    OLED_Clear();
13    OLED_Refresh();
14 }
15
16 void OLED_Refresh(void)
17 {
18     OLED_Cmd(0x22);
19     HAL_I2C_Master_Transmit(&hi2c1, OLED_ADDR, OLED_Buffer, 128*8,
20                             100);
21 }
22 // 支持字符、字符串、中文、大数字、绘图等函数
```

Listing 7: OLED 驱动核心函数

bt.c — 蓝牙通信

```
1 #define BT_BAUDRATE  9600
2
```

```
3 void BT_Init(void)
4 {
5     HAL_UART_Transmit(&huart1, (uint8_t*)"AT+NAME=SportsWatch\r\n",
6         ...);
7 }
8 void BT_Upload(void)
9 {
10     char buffer[128];
11     snprintf(buffer, sizeof(buffer), "%d, %.2f, %.1f, %02d:%02d\n",
12         stepCount, distance, calorie, hour, minute);
13     HAL_UART_Transmit(&huart1, (uint8_t*)buffer, strlen(buffer),
14         100);
15 }
```

Listing 8: 蓝牙初始化和数据上传

encoder.c — 旋转编码器驱动

```
1 typedef enum {
2     EVENT_NONE,
3     EVENT_ROTATE_CW,
4     EVENT_ROTATE_CCW,
5     EVENT_BUTTON_PRESS,
6     EVENT_BUTTON_LONG_PRESS
7 } EncoderEvent_t;
8
9 void Encoder_Poll(void)
10 {
11     // 读取A/B相，软件消抖，状态机判断方向，按键检测
12 }
```

Listing 9: 编码器事件枚举与轮询

3.2.4 技术实现细节

表 2: 硬件资源分配

模块	接口	引脚
OLED	I2C1	PB6(SCL), PB7(SDA)
MPU6050	I2C1	PB6(SCL), PB7(SDA)
蓝牙	USART1	PA9(TX), PA10(RX)
编码器	GPIO	PA0(A), PA1(B), PA2(按键)

表 3: FreeRTOS 任务

任务	功能	周期
StartDefaultTask	初始化各模块，自删除	仅一次
Task_Main	主循环：输入 → 处理 → 输出	10ms

核心模块实现要点：

- **UI 状态机**：共 6 个状态，通过编码器切换。
- **计步算法**：合加速度 $mag = \sqrt{ax^2 + ay^2 + az^2}$ ，双沿滞回检测（高 1.2g，低 0.8g），180ms 最小间隔，2 秒超时复位。
- **OLED 显示**：1KB 双缓冲，批量 I2C 刷新，支持多种字体。
- **编码器**：轮询 +5ms 消抖，检测旋转、短按、长按。
- **蓝牙**：每 2 秒上传“步数, 距离, 卡路里, 温度”，支持 RESET 清零。

表 4: 资源占用

资源	占用	总量
Flash	~31.5 KB	64 KB
RAM	~14.9 KB	20 KB
主循环周期	10 ms	-

开发工具：Keil MDK-ARM v5 + ST-Link V2 + FreeRTOS + STM32F1xx HAL 库。

4 物料清单与项目成本

4.1 物料清单

表 5: 物料清单

序号	名称	型号/规格	数量	单价 (元)
1	STM32 主控	STM32F103C8T6 最小系统板	1	~12
2	OLED 屏	1.3 寸 I2C SH1106	1	~9
3	六轴传感器	MPU6050 模块	1	~4
4	蓝牙模块	HC-05 主从机一体	1	13.87
5	旋转编码器	EC11 (带按键)	1	~1
6	锂电池	602030 3.7V 300mAh	1	4.41
7	充电模块	5V 充放电一体模块	1	3.11
8	杜邦线/排针	若干	-	~3

4.2 项目总成本

表 6: 项目总成本

项目	金额 (元)
蓝牙模块 HC-05	13.87
锂电池 602030 300mAh	4.41
充电模块 5V 充放一体	3.11
其他模块	~29
总计	50 元

4.3 损耗与损坏记录

表 7: 损耗与损坏记录

项目	数量	备注
元器件损坏	0	无损坏
模块损耗	0	无损耗

说明：本项目采用模块化器件，全部物料采购总成本约 50 元，组装与调试过程中无任何损坏。

5 个人贡献

本人作为团队核心成员，全程参与了本项目的需求分析、方案设计、代码编写、硬件搭建及文档撰写工作，具体贡献如下：

1. 报告撰写分工

本人负责了本报告大部分技术内容的撰写与统稿工作，主要包括：

- 计步算法模块的详细技术说明，包括算法原理、实现流程以及参数调优过程的完整描述；
- 软件设计章节中任务划分、关键代码逻辑、UI 状态机架构等内容的系统梳理与文字表达；
- 硬件连接部分的整理与校对，确保引脚分配、模块接口等信息的准确性；
- 全文的格式统一、语言润色及逻辑连贯性优化。

2. 个人编码工作内容

- **计步算法实现**：基于 MPU6050 原始数据，独立编写合加速度计算、双沿滞回状态机、滤波及复位等核心代码，并负责参数调优。
- **UI 界面开发**：参与运动页面的数据更新与显示逻辑编写，实现步数、距离、卡路里的实时刷新，协助完成 RTC 时间显示。
- **传感器驱动开发**：参与 MPU6050 驱动代码的编写与调试，包括 I2C 读写、寄存器配置、数据校准及温度读取。
- **代码维护与联调**：协助调试编码器事件与计步数据的联动，解决时序冲突，参与系统整合。

3. 硬件连接工作

- 参与模块布局规划，协助确定 MPU6050 最佳安装位置以减少噪声干扰。
- 协助完成 OLED、MPU6050、蓝牙模块的 I2C 和 UART 接线，并检查电源极性。

4. 资本投入

本项目硬件物料费用由团队成员平摊，个人承担部分约 25 元。

5. 开发中遇到的问题及解决方案

表 8: 问题与解决方案

问题	解决方案
静止时误计步	增加 2 秒静止超时自动复位状态机
高频抖动导致步数虚增	增加 180ms 最小间隔滤波
步行时漏计步	调整滞回阈值（拍臂 45°/放臂 10°）
加速度方向偏差	改用 $\text{atan2}(ax, -ay)$ 计算角度

6 项目成果与反思

6.1 成果展示

成功完成基于 STM32F103C8T6 的运动手表原型开发，实现了以下功能：

- **OLED 双页面显示：**时间页面显示日期、大字体时间及温度；运动页面显示步数、距离、卡路里，页面切换流畅。
- **计步算法：**基于 MPU6050 加速度数据的双沿滞回状态机，在日常步行场景下计步准确率良好。
- **蓝牙数据上传：**手表每 2 秒向上位机发送运动数据，通信稳定无丢包。
- **PC 端上位机：**配套 Python 程序通过 BLE 实时接收并显示手表数据，支持历史记录保存和清零指令下发。

6.2 项目评估

成功之处：系统整体功能完整，从硬件搭建、嵌入式开发到上位机实现全链路打通；编码器交互逻辑清晰，操作体验友好；模块化器件方案成本低（约 50 元），适合原型验证和教学演示。

不足之处：计步算法在慢走、快跑等不同步态下的适应性有待提升；设备未实现低功耗休眠模式，电池续航有限；运动数据仅在 RAM 中保存，掉电后丢失；蓝牙功能目前仅支持单向上传，双向控制协议尚不完善。

6.3 个人反思与改进方向

本次项目中计步算法调优花费时间最多。初期因对传感器噪声特性和步态变化规律理解不足，导致计步结果波动较大。通过反复采集数据、调整阈值，逐步掌握了状态机设计的关键技巧，认识到嵌入式算法必须与实际物理场景紧密结合。

后续改进方向：

- 引入自适应阈值或轻量级机器学习模型，提升不同步态下的鲁棒性；
- 增加外部 Flash 实现数据持久化；
- 增加低功耗休眠模式延长续航；
- 完善蓝牙双向通信协议，支持更多远程控制功能。

7 附录

7.1 完整代码清单

7.1.1 应用层代码（项目核心源文件）

表 9: 应用层源文件列表

文件	路径	说明
main.c	Core/Src/main.c	主函数，系统初始化入口
main.h	Core/Inc/main.h	主头文件
freertos.c	Core/Src/freertos.c	FreeRTOS 任务创建与管理
ui.c	Core/Src/ui.c	UI 状态机（6 个页面）
ui.h	Core/Inc/ui.h	UI 头文件
oled.c	Core/Src/oled.c	OLED 128×64 I2C 驱动
oled.h	Core/Inc/oled.h	OLED 头文件
bt.c	Core/Src/bt.c	蓝牙模块驱动（UART）
bt.h	Core/Inc/bt.h	蓝牙头文件
pedometer.c	Core/Src/pedometer.c	计步算法
pedometer.h	Core/Inc/pedometer.h	计步头文件
mpu6050.c	Core/Src/mpu6050.c	加速度计/陀螺仪驱动
mpu6050.h	Core/Inc/mpu6050.h	MPU6050 头文件
encoder.c	Core/Src/encoder.c	旋转编码器驱动
encoder.h	Core/Inc/encoder.h	编码器头文件
rtc.c	Core/Src/rtc.c	RTC 实时时钟驱动

表 9 – 继续

文件	路径	说明
rtc.h	Core/Inc/rtc.h	RTC 头文件
usart.c	Core/Src/usart.c	USART1 串口初始化
usart.h	Core/Inc/usart.h	USART 头文件
i2c.c	Core/Src/i2c.c	I2C 初始化
i2c.h	Core/Inc/i2c.h	I2C 头文件
gpio.c	Core/Src/gpio.c	GPIO 引脚初始化
gpio.h	Core/Inc/gpio.h	GPIO 头文件
stm32f1xx_it.c	Core/Src/stm32f1xx_it.c	中断服务函数
stm32f1xx_it.h	Core/Inc/stm32f1xx_it.h	中断头文件
stm32f1xx_hal_msp.c	Core/Src/stm32f1xx_hal_msp.c	HAL MSP 初始化
cn_font.h	Core/Inc/cn_font.h	中文字库
large32.h	Core/Inc/large32.h	大数字字体
FreeRTOSConfig.h	Core/Inc/FreeRTOSConfig.h	FreeRTOS 配置
stm32f1xx_hal_conf.h	Core/Inc/stm32f1xx_hal_conf.h	HAL 库配置

7.1.2 启动文件

表 10: 启动文件

文件	路径	说明
startup_stm32f103xb.s	MDK-ARM/startup_stm32f103xb.s	STM32F103C8 启动文件（汇编）

7.1.3 HAL 驱动层 (STM32F1xx HAL 库)

表 11: HAL 驱动源文件 (路径: Drivers/STM32F1xx_HAL_Driver/Src/)

文件
stm32f1xx_hal.c
stm32f1xx_hal_rcc.c / stm32f1xx_hal_rcc_ex.c
stm32f1xx_hal_gpio.c / stm32f1xx_hal_gpio_ex.c
stm32f1xx_hal_i2c.c
stm32f1xx_hal_uart.c
stm32f1xx_hal_rtc.c / stm32f1xx_hal_rtc_ex.c
stm32f1xx_hal_cortex.c
stm32f1xx_hal_dma.c
stm32f1xx_hal_flash.c / stm32f1xx_hal_flash_ex.c
stm32f1xx_hal_pwr.c
stm32f1xx_hal_exti.c

7.1.4 FreeRTOS 实时操作系统

表 12: FreeRTOS 源文件 (路径: Middlewares/Third_Party/FreeRTOS/Source/)

文件
tasks.c
queue.c
timers.c
event_groups.c
stream_buffer.c
list.c
croutine.c
heap_4.c
port.c (portable/RVDS/ARM_CM3/)
cmsis_os2.c (CMSIS_RTOS_V2/)

7.1.5 CMSIS 标准库

表 13: CMSIS 文件

文件	说明
core_cm3.h	Cortex-M3 内核寄存器定义
cmsis_compiler.h	编译器抽象层
cmsis_gcc.h / cmsis_armcc.h	编译器适配
system_stm32f1xx.c / .h	系统时钟初始化
stm32f103xb.h	STM32F103 外设寄存器定义
stm32f1xx.h	STM32F1 系列总头文件

7.1.6 项目配置文件

表 14: 项目配置文件

文件	路径	说明
SportsWatch.uvprojx	MDK-ARM/SportsWatch.uvprojx	Vision 项目文件
SportsWatch.uvoptx	MDK-ARM/SportsWatch.uvoptx	项目选项配置
SportsWatch.sct	MDK-ARM/SportsWatch/SportsWatch.sct	分散加载文件
RTE_Components.h	MDK-ARM/RTE/_SportsWatch/RTE_Components.h	运行环境配置
SportsWatch.ioc	SportsWatch.ioc	STM32CubeMX 配置
JLinkSettings.ini	MDK-ARM/JLinkSettings.ini	JLink 调试器设置